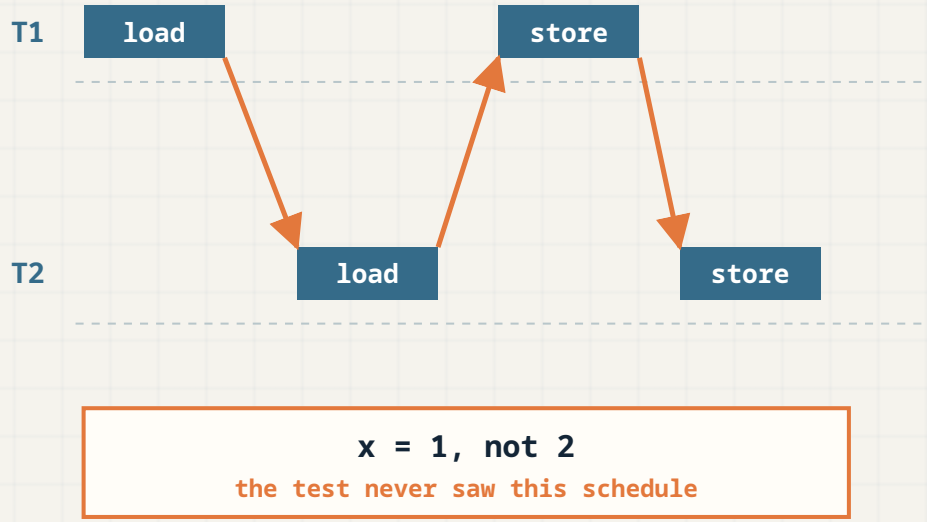
PRACTICE SPECIMEN · CONCURRENCY

TWO THREADS / ONE BAD SCHEDULE

concurrency bugs

and the tools that find them



One question for 90 minutes

A test checks one schedule. The bug lives in another.

How do you find the bad schedule before production does?

> SCHEDULE
>
> **The order in which the scheduler interleaves the steps of threads. The OS picks it, not you.**

01 **What bad schedules look like**
data race, check-then-act, deadlock, livelock, starvation, convoy

02 **Who finds them**
from the compiler to production: static analysis, TSan, injection, exhaustive search, Lincheck, TLA+, JFR

03 **Quiz: the same bugs in Go, Loom, Node**
recognise the classics in new syntax

04 **The AWS postmortem and five actions**
the October 2025 outage and what to do this week

A green test

RACE.GO GO 1.27

```go
func main() {
    var wg sync.WaitGroup
    c := 0
    for g := 0; g < 2; g++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for k := 0; k < 2000; k++ {
                c++
            }
        }()
    }
    wg.Wait()
    fmt.Println("c =", c, "(expected 4000)")
}
```

QUESTION

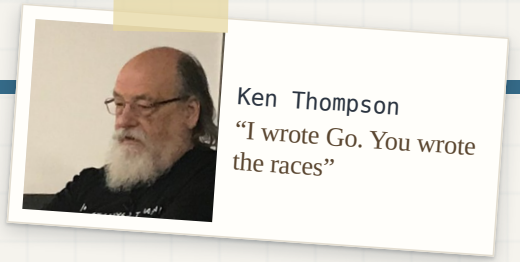
We run it 10 times. How many times do we get exactly 4000?

Almost always: **about 97%** of runs. Each goroutine finishes its 2000 steps within one time slice.

RECORDING

```
$ for i in 1 2 3; do go run race.go; done
c = 4000 (expected 4000)
c = 4000 (expected 4000)
c = 4000 (expected 4000)
```

The detector sees what the test does not



THE SAME RUN

The answer is correct: 4000.

Races found: two.

WHAT -RACE CHECKS

Not the result but the order of memory accesses. Two writes to `c` with no synchronization between them mean that a schedule exists where they collide.

Go docs, “Data Race Detector”

RECORDING

```
$ go run -race race.go
WARNING: DATA RACE
Read at 0x00c000188038 by goroutine 9:
      race.go:16 +0x99
Previous write at 0x00c000188038 by goroutine 8:
      race.go:16 +0xab
      race.go:13 +0x87
      race.go:13 +0x87
WARNING: DATA RACE
Write at 0x00c000188038 by goroutine 9:
      race.go:16 +0xab
Previous write at 0x00c000188038 by goroutine 8:
      race.go:16 +0xab
      race.go:13 +0x87
      race.go:13 +0x87
c = 4000 (expected 4000)
Found 2 data race(s)
```

A thread-safe map, unsafe code

8 THREADS × 100,000
ONE KEY

CHECKTHENACT.JAVA · THREAD LOOP — JDK 21

```
ConcurrentHashMap<String,Integer> m = …;

for (int k = 0; k < N; k++) {
    if (fix) m.merge("hits", 1, Integer::sum);
    else {
        Integer v = m.get("hits");          // check
        m.put("hits", v==null?1:v+1);   // act
    }
}
```

RECORDING

```
$ java CheckThenAct
fix=false expected=800,000 actual=199,625 lost=600,375
```

RECORDING

```
$ java CheckThenAct fix
fix=true expected=800,000 actual=800,000 lost=0
```

QUESTION

How many of the 800,000 increments survive: all, almost all, half, a quarter?

About a quarter: on 28, 8 and 4 cores 69–80% are lost. Between `get` and `put` another thread reads the same value.

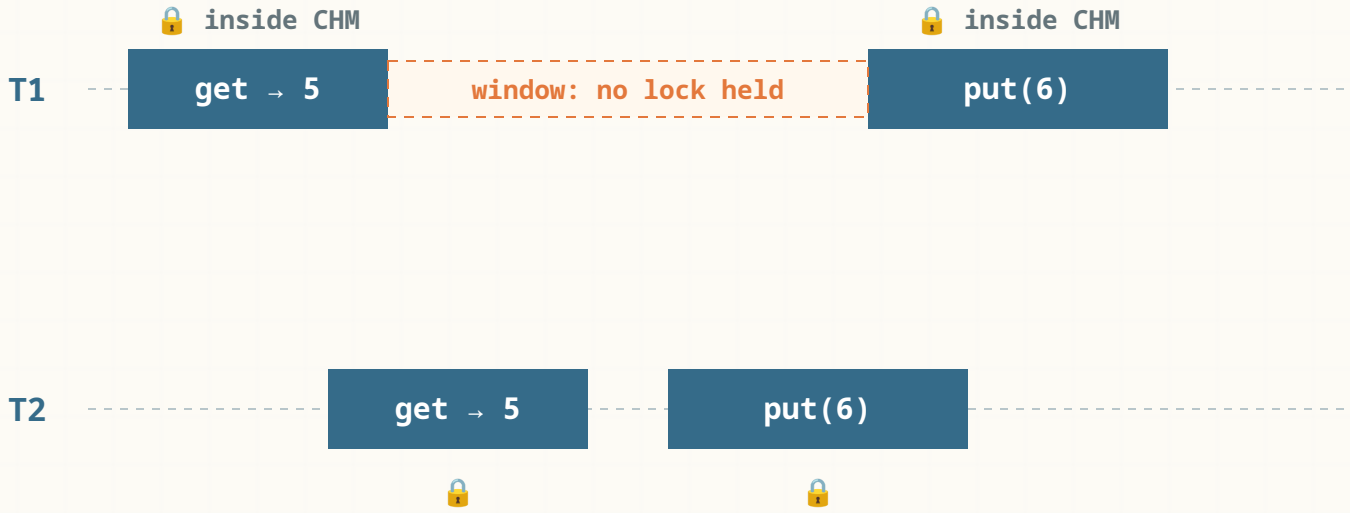


James Gosling
"Java is thread-safe. Your get+put was never in the deal"

Why the race detector stays silent

01

TWO THREADS, ONE MAP



two put(6) instead of 6 and 7

every access is synchronized → nothing for the detector to report

97%

of non-deadlock bugs are atomicity or order violations

105 real bugs in MySQL, Apache, Mozilla, OpenOffice

Lu et al., ASPLOS'08, "Learning from Mistakes"

TWO TERMS

atomicity violation — a pair of operations had to run without interference.
order violation — B had to happen after A, but happened before.

QUESTION

Will `-race` or TSan catch this loss?

No. They look for accesses without happens-before, and here every access is under a lock.

A deadlock jstack finds on its own

LOCKORDER.JAVA JDK 21

```
static final Object A = new Object(), B = new Object();

new Thread(() -> {
    synchronized (A) { sleep(50);
        synchronized (B) {} }
}, "transfer-A-to-B");

new Thread(() -> {
    synchronized (B) { sleep(50);
        synchronized (A) {} }
}, "transfer-B-to-A");
```

WHAT IS NEW

jstack builds the “who waits for whom” graph over monitors by itself and prints the cycle.

RECORDING

```
$ bash lo.sh
Found one Java-level deadlock:
=============================
"transfer-A-to-B":
  waiting to lock monitor 0x00007f5814001d80 (object 0x000000062e0143a0, a java.
lang.Object),
  which is held by "transfer-B-to-A"

"transfer-B-to-A":
  waiting to lock monitor 0x00007f5808000f50 (object 0x000000062e014390, a java.
lang.Object),
```

A deadlock in a thread pool

4 THREADS IN THE POOL
4 TASKS WAIT FOR SUBTASKS

POOLDEADLOCK.JAVA — JDK 21

```
pool = Executors.newFixedThreadPool(4);
for (int i = 0; i < 4; i++) pool.submit(() -> {
    Future<String> sub = pool.submit(() -> "page-"+id); // queued
    sub.get();                                          // waits forever
    …
```

POOL: 4 SLOTS

req 0 get()	req 1 get()	req 2 get()	req 3 get()

POOL QUEUE

sub0	sub1	sub2	sub3

tasks hold every slot and wait for subtasks;
subtasks need a free slot — there is none

QUESTION

jstack just found a deadlock. Will it find this one?

No: “Found one Java-level deadlock” lines — 0. A deadlock can happen on any bounded resource, not only on a lock.

RECORDING

```
$ bash pool.sh
main: still waiting... (try jstack)
--- pool threads:
"pool-1-thread-1"
   WAITING (parking)
    at java.util.concurrent.FutureTask.get
"pool-1-thread-2"
   WAITING (parking)
    at java.util.concurrent.FutureTask.get
"pool-1-thread-3"
   WAITING (parking)
    at java.util.concurrent.FutureTask.get
"pool-1-thread-4"
   WAITING (parking)
    at java.util.concurrent.FutureTask.get
--- 'Found one Java-level deadlock' lines in jstack: 0
```



Edsger Dijkstra
“I invented semaphores. The unbounded pool is on you”

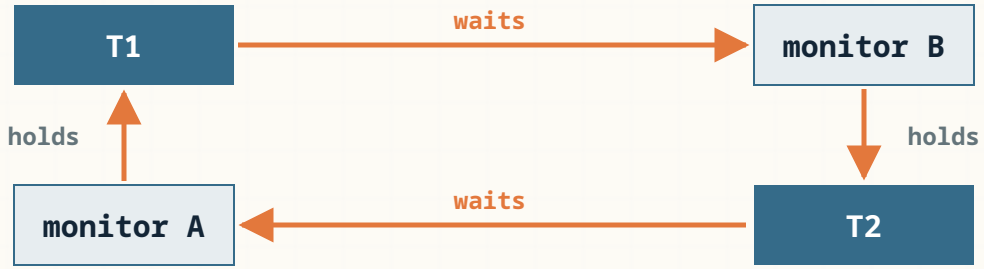
NO OWNER → NO EDGE
NO EDGE → NO CYCLE

What jstack sees

01

HAS AN OWNER · JSTACK SEES IT

- a `synchronized` monitor
- `ReentrantLock` and other `AbstractOwnableSynchronizer`s

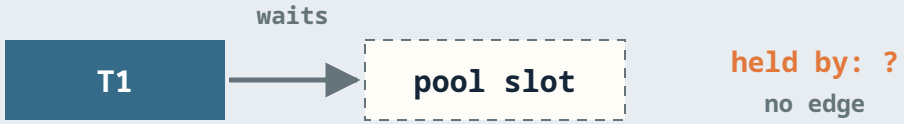


The JVM knows who holds each such lock. That gives "waits → holds" edges, and jstack prints a cycle of them as "Found one Java-level deadlock".

02

NO OWNER · JSTACK IS BLIND

- a thread-pool slot, `Future.get()`
- `Semaphore`: a permit belongs to nobody
- a connection from a DB pool
- a channel, a queue, a network reply



Threads hang in WAITING, but the JVM never recorded whom they wait for. The cycle exists only in your head.

Deadlock, livelock, starvation

	SYSTEM PROGRESS?	INDIVIDUAL THREAD?	CPU	JSTACK	RESOLVES ON ITS OWN?
deadlock	no	no	~0%	BLOCKED / WAITING, stacks unchanged	never
livelock	steps occur, no useful work	no	high	RUNNABLE, stacks change	with another schedule, perhaps
starvation	yes	one does not	any	nothing distinctive	possibly

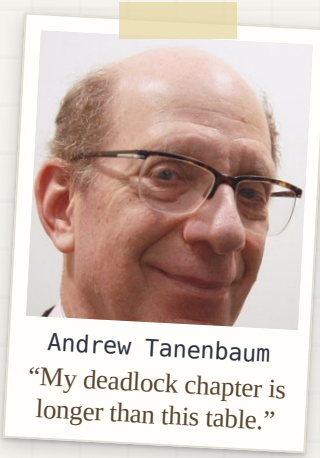
DEADLOCK

A special case of starvation: everyone is stuck.

LIVELOCK

Even a fair scheduler can let everyone run without progress.

Compare Stanford CS149 and Berkeley CS162



Andrew Tanenbaum
"My deadlock chapter is longer than this table."

Fixed deadlock, got livelock

deadlock
each holds one lock and waits for the other

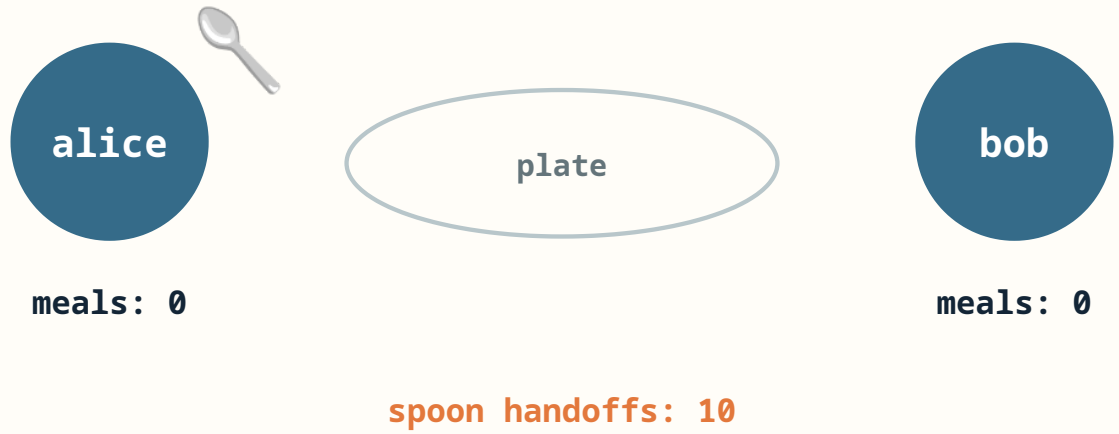
→

“Can't get in? Step aside.”
release your lock and try again

→

livelock
both step aside at the same time, forever

MODEL



RULE

One spoon. If the other diner is hungry, pass it over.

Both stay hungry: the spoon keeps moving, but nobody starts eating.

OSTEP, ch. 32; Cambridge, Concurrent and Distributed Systems

Polite diners: 0 meals, 200% CPU

RECORDING

```
$ bash livelock.sh
t=1s  alice ate 0  bob ate 0   spoon handovers 19,133,702
--- jstack:
"alice" State: RUNNABLE
"bob"   State: RUNNABLE
jstack: no deadlock reported
--- top:
    PID %CPU CMD
2276550  199 java PoliteCouple
t=2s  alice ate 0  bob ate 0   spoon handovers 19,295,323
t=3s  alice ate 0  bob ate 0   spoon handovers 19,205,222
t=4s  alice ate 0  bob ate 0   spoon handovers 19,311,228
```

QUESTION

Nobody eats. Will CPU usage be 0%, 100%, or 200%? Will jstack show BLOCKED or RUNNABLE?

199% and RUNNABLE. Two cores are busy passing the spoon: about 19 million handoffs per second.

WHO NOTICES?

CPU usage — no: the process is busy.
jstack — no: both threads are RUNNABLE; no deadlock.
Meal counter — yes: 0.

Breaking a livelock

POLITECOUPLE.JAVA · DINER() — FIX = STOP YIELDING

```
if (hungry[1-me].get() && !fix) {    // "after you"
    spoon = other; handovers.increment(); continue;
}
meals[me].increment();                // spoon owner eats
spoon = other;                           // then passes it
```

RECORDING

```
$ java PoliteCouple fix
t=1s  alice ate 4,413,438  bob ate 4,413,533   spoon handovers 0
t=2s  alice ate 4,035,776  bob ate 4,036,208   spoon handovers 0
t=3s  alice ate 3,949,739  bob ate 3,949,231   spoon handovers 0
t=4s  alice ate 3,988,803  bob ate 3,988,802   spoon handovers 0
```

01

Priority

Whoever has the spoon eats before passing it. That is the fix on the left.

02

Queue

Arrival order decides whose turn it is; politeness does not.

03

Randomized delay (jitter)

Wait a random interval before retrying, so the threads drift out of phase. The same applies to retries.

A polite spinlock

POLITE SPINLOCK — C = 0

```
void Lock() {
    while (c.fetch_add(1) > 0) // someone here?
        c.fetch_sub(1);        // back off
}
void Unlock() { c.fetch_sub(1); }
```

RECORDING

```
often = FAIR) ----
  B sub                c=1
  B add(old=1)         c=2
  A sub                c=1
  A add(old=1)         c=2
  B sub                c=1
  B add(old=1)         c=2
  A sub                c=1
  A add(old=1)         c=2
  B sub                c=1
  B add(old=1)         c=2
  A sub                c=1
  A add(old=1)         c=2
fuel 1,000,000 steps spent: critical sections entered 0,
counter c=2 (nobody holds the lock!)
```

Same protocol in Java. Livelock exercise from R. Lipovsky's course.

MODEL

2	1,000,017	0
c (counter)	steps	CS entries in BBAA loop

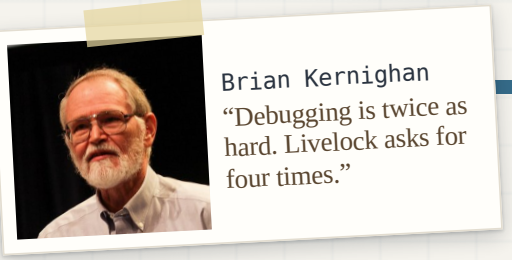
A · back off: sub

B · back off: sub

```
---- repeat BBAA forever ----
  B sub                c=1
  B add(old=1)         c=2
  A sub                c=1
  A add(old=1)         c=2
  B sub                c=1
  B add(old=1)         c=2
  A sub                c=1
  A add(old=1)         c=2
  B sub                c=1
  B add(old=1)         c=2
  A sub                c=1
  A add(old=1)         c=2
another 1,000,000 BBAA steps: CS entries 0, c=2 (nobody holds the lock)
```

One million steps of a fair schedule: 0 entries, c=2. Both see old=1 and back off.

Random schedules miss this livelock



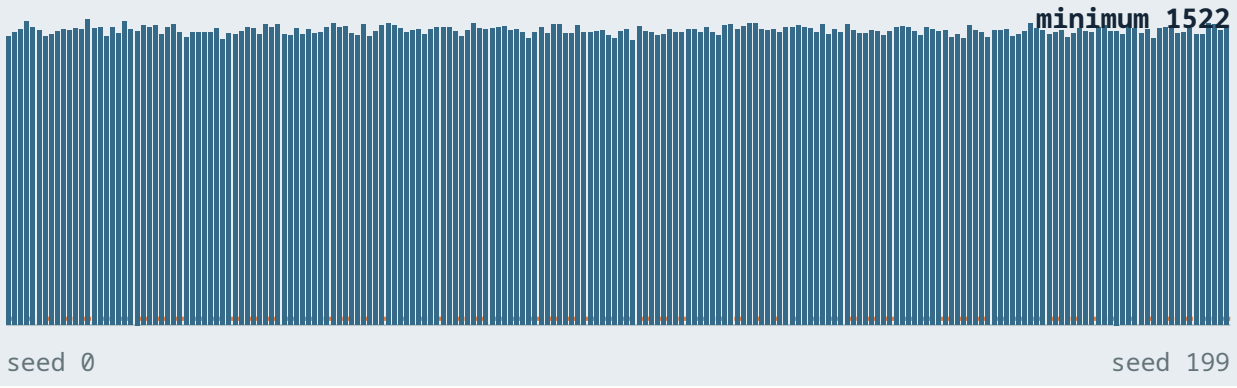
Brian Kernighan
“Debugging is twice as hard. Livelock asks for four times.”

200 SEEDS × 10,000 STEPS
RANDOM ≠ ADVERSARY

MODEL

200	1522	1637
seeds run	fewest CS entries	most CS entries

dashed baseline: 0 entries, as with BBAA



RECORDING

```
$ java EvilScheduler spin random 200
200 random seeds x 10,000 steps: worst seed still entered CS 1522 times -> random never finds the livelock
```

QUESTION

Will 200 random schedules of 10,000 steps find this livelock?

No. Even the worst seed entered the critical section 1,522 times. This livelock needs a precise interleaving; randomness breaks it.

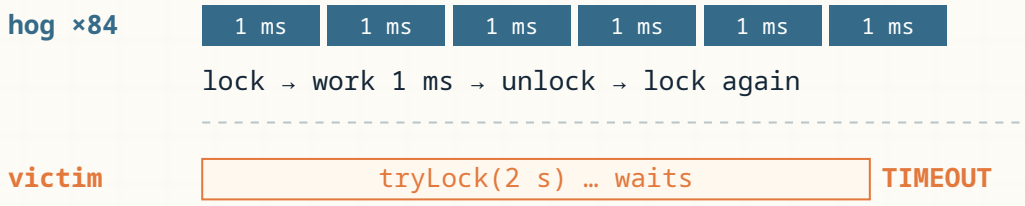
IN PRACTICE

Livelock often looks like low throughput, not exactly zero: tryLock with rollback slowed down on 28 cores but still made progress. Track useful work; the checker in Part 2 finds the cycle.

Starvation

01

STARVE2.JAVA



lock released ~1,000 times per second
greedy threads = 3 × cores (28 → 84),
leaving no spare core for the victim

UNLIKE DEADLOCK OR LIVELOCK

Deadlock: everyone stops. Livelock: everyone runs, nobody advances. Starvation: the system advances, but one thread does not.

RECORDING

```
$ java Starve2
fair=false hogs=84 victim: TIMEOUT TIMEOUT TIMEOUT (got lock 0 times)
```

WHAT YOU OBSERVE

The system completes thousands of lock acquisitions per second. One thread makes no progress. Average throughput hides it.

QUESTION

In three 2-second attempts, how many times will the victim acquire the lock?

None. The lock becomes free thousands of times per second, but someone else takes it each time.

Barging and fair locks

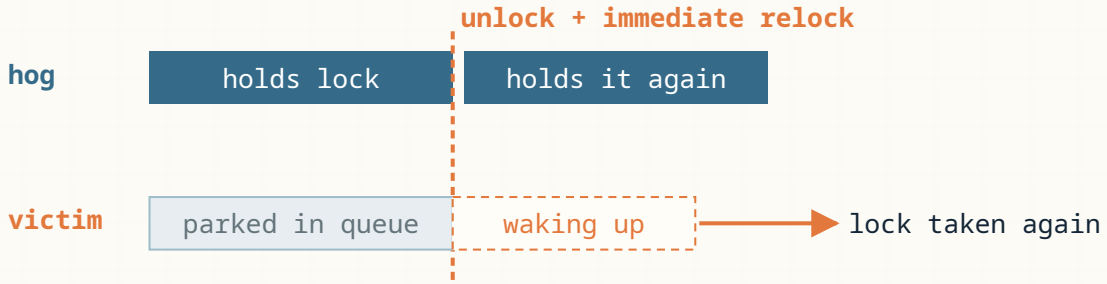
UNFAIR: WHOEVER IS ON CPU WINS
FAIR: FIFO



Butler Lampson
"notify is a hint, not a promise."

01

WHY THE VICTIM ARRIVES TOO LATE



Unparking takes microseconds; re-locking can take nanoseconds.

A FIFO lock hands off to the queue head, not whoever is on CPU.

IMPORTANT

`tryLock(timeout)` reveals starvation but does not cure it. Try FIFO, aging, or a shorter critical section.

RECORDING

```
$ java Starve2 fair
fair=true hogs=84 victim: 92ms 93ms 92ms 92ms 91ms 91ms 91ms 91ms 9
2ms 91ms (got lock 10 times)
```

THE SAME 84 THREADS, REENTRANTLOCK(TRUE)

The victim acquires the lock 10/10 times, after about 92 ms each time: roughly one queue round with 84 threads holding it for 1 ms.

Default mutexes and semaphores need not be FIFO (NUS CS3211). Under Mesa monitor semantics, a woken thread need not acquire the monitor first (Lampson & Redell, 1980).

The cost of fairness

RECORDING

```
$ java Convoy unfair; java Convoy fair
unfair threads=8  ops/s=  1,387,390   context switches per lock acquisition = 0.20
fair   threads=8  ops/s=    314,729   context switches per lock acquisition = 1.00
```

1,387,390
unfair · acquisitions per second

314,729
fair · acquisitions per second

0.20
unfair · context switches per acquisition

1.00
fair · context switches per acquisition

Each thread reads voluntary_ctxt_switches from /proc/thread-self/status. Across 28, 8 and 4 cores, throughput differs by 3.6–5×; the fair lock measured 1.00 in all 30 runs.

QUESTION

With a fair lock for 8 identical threads, will throughput stay the same, fall by 10%, by 50%, or by several times?

4.4× lower.

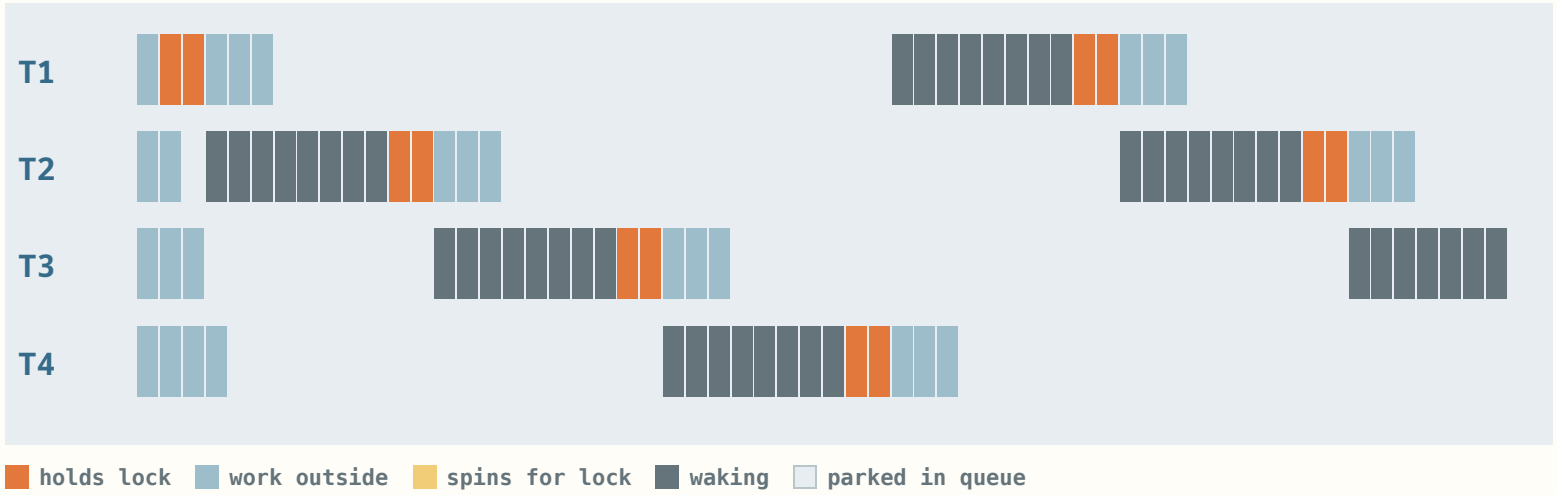
WHY EXACTLY 1.00?

FIFO hands the lock to a parked thread. Each acquisition requires a wake-up and a context switch.

Lock convoy

FIFO → LOCK GOES TO A PARKED THREAD
THE QUEUE MOVES AT WAKE-UP SPEED

MODEL · 4 THREADS



6
lock acquisitions

6
wake-ups

1.00
wake-ups per acquisition

MECHANISM

Unfair: a running thread can barge; fewer threads sleep. Fair: the lock goes to the queue head, which is parked. Each handoff waits for a wake-up.

8 THREADS, 3 S

	ACQUISITIONS	MIN / MAX SHARE
unfair	3 508 757	0,924
fair	756 857	0,997

TAKEAWAY

Pay for fairness only when bounded waiting matters. Do not enable it by default.

How tools search for bugs

MORE CONTROL OVER SCHEDULES →

L1 static

who chooses
nobody: no execution

runs
0

bug
declared locking discipline broken

L2 dynamic

who chooses
OS

runs
1

bug
conflicting accesses without happens-before; lock-order cycle

`TSan · go -race`

L3 scheduling

who chooses
you: injection, seed, PCT, exploration

runs
many

bug
invariant violated

`yield/sleep · sim.py`

L4 checkers

who chooses
the tool's scheduler

runs
all within a bound

bug
no valid sequential ordering explains the result; forbidden outcome

`Lincheck · jcstress`

L5 model

who chooses
none: state-space exploration

runs
all model states

bug
invariant violated or liveness violated

`TLA+ · DST`

L6 production

who chooses
OS and users

runs
actual runs

bug
waiting, leaks, no progress

`jstack · JFR · pprof`

withdraw: V0, V1, V2

V0 · NO LOCK — BANK.CPP

```
if (balance >= amt) {
  window();
  balance -= amt;
  return true;
}
return false;
```

V1 · TWO CRITICAL SECTIONS — BANK.CPP

```
bool ok;
// check
{ Guard g(mu); ok = balance >= amt; }
window();
// act
if (ok) { Guard g(mu); balance -= amt; }
return ok;
```

V2 · ONE CRITICAL SECTION — BANK.CPP

```
Guard g(mu);
if (balance < amt) return false;
window();
balance -= amt;
return true;
```

V1

Every access to `balance` holds the lock. The check and debit use separate critical sections: both threads can pass the check, then both debit the account.

TWO MORE CASES FROM PART 1

`transfer(a, b)` — for deadlock; the polite pair — for livelock.

QUESTION

Which of the six tool levels will catch V1?

Make your predictions. We will check them against the summary table.

Clang Thread Safety Analysis

TSA.H · BANK.CPP …

```
class CAPABILITY("mutex") Mutex {
  std::mutex m_;
 public:
  void lock()   ACQUIRE() { m_.lock(); }
  void unlock() RELEASE() { m_.unlock(); }
};
…
class Account {
  Mutex mu;
  long balance GUARDED_BY(mu) = 100;
```

WHAT IT CHECKS

`GUARDED_BY(mu)`: access this field only while holding `mu`. `REQUIRES(mu)` specifies a function's lock precondition. Analysis is local to a function.

PLAIN STD::MUTEX IS NOT ENOUGH

libstdc++'s `std::mutex` lacks thread-safety annotations: Clang warns about `GUARDED_BY` itself. Use an annotated wrapper or `absl::Mutex`.



Bjarne Stroustrup
"C++ lets you do anything. Including a race without a warning."

Clang docs, «Thread Safety Analysis»; Abseil uses ABSL_GUARDED_BY

RECORDING

```
$ clang++ -std=c++20 -fsyntax-only -Wthread-safety -Werror -DV=0 bank.cpp
bank.cpp:33:9: error: reading variable 'balance' requires holding mutex 'mu' [-Werror,-Wthread-safety-analysis]
   33 |     if (balance >= amt) {          // clang: reading 'balance' requires holding mutex 'mu'
      |         ^
bank.cpp:35:7: error: writing variable 'balance' requires holding mutex 'mu' exclusively [-Werror,-Wthread-safety-analysis]
   35 |       balance -= amt;              // clang: writing 'balance' requires holding mutex 'mu'
      |       ^
2 errors generated.
```

V1 passes static analysis

QUESTION

V1 checks and debits under a lock, but in separate critical sections. Will the compiler warn?

No. Every access to `balance` holds the lock. The bug is in the gap between sections.

ACCOUNT.JAVA · ERRORPRONE …

```
@GuardedBy("mu") private long balance = 100;

public boolean withdraw(long amt) {
  if (balance < amt) return false;  // no mu
  synchronized (mu) { balance -= amt; }
  return true;
}
public boolean withdrawSplit(long amt) {  // V1
  boolean ok;
  synchronized (mu) { ok = balance >= amt; }
  if (ok) { synchronized (mu) { balance -= amt; } }
  return ok;
}
```

RECORDING

```
$ clang++ … -Wthread-safety -Werror -DV=1 bank.cpp && echo OK
OK
```

MVN COMPILE · ERRORPRONE · PATH SHORTENED

```
Account.java:[9,9] [GuardedBy] This access should be guarded by 'this.mu', which is not currently held
```

Only **withdraw** triggers a warning. **withdrawSplit** (V1) in the same file passes.

WHAT L1 MISSES

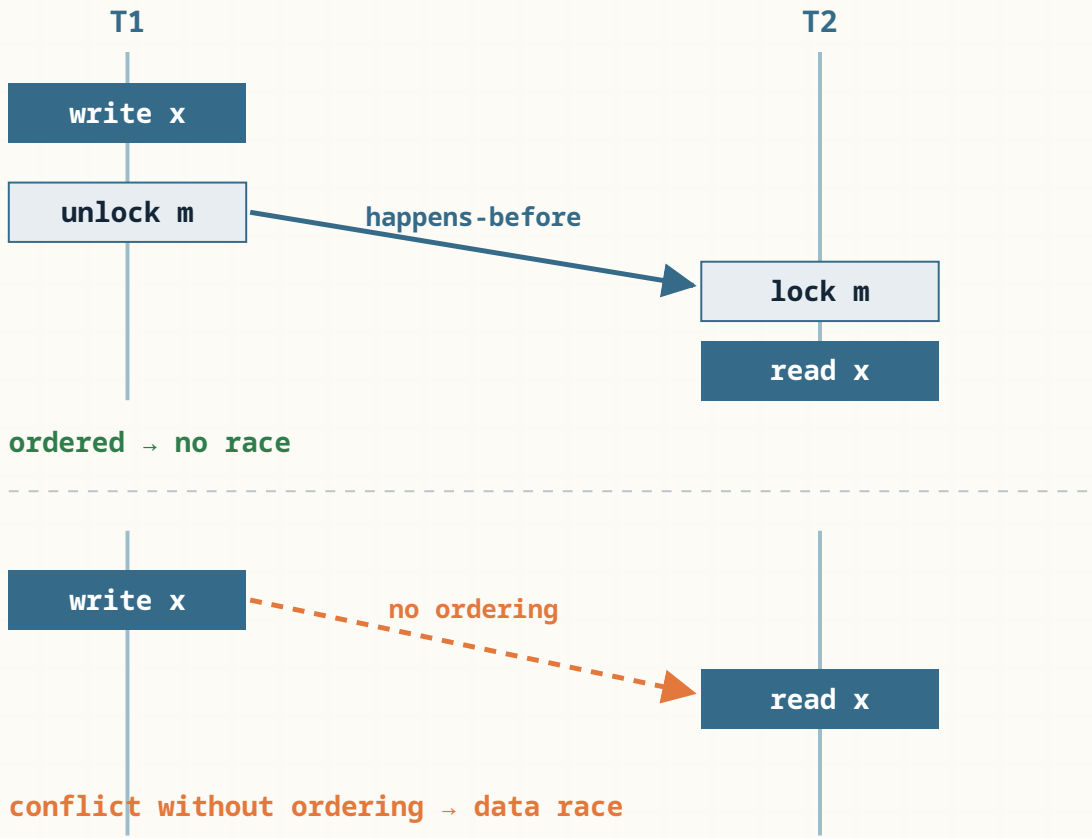
Atomicity violations (V1), lock-free code, argument-dependent lock order, and code without annotations.

Related checks: IntelliJ inspections, SpotBugs IS2_INCONSISTENT_SYNC, RacerD. Rust's Send/Sync prevents data races, but a Mutex-based V1 still compiles.

How a race detector recognizes a race

01

HAPPENS-BEFORE



Lockset · Eraser, 1997

Warns when accesses share no protecting lock. Less dependent on timing, but can flag synchronization that uses joins, queues, or atomics instead of locks.

Happens-before · FastTrack 2009, TSan

Few false positives, but only checks executed paths. Threads and synchronization objects carry vector clocks; memory locations record the latest epoch. TSan does not maintain locksets.

COST

	TIME	MEMORY
C++ TSan	×5–15	×5–10
Go -race	×2–20	×5–10

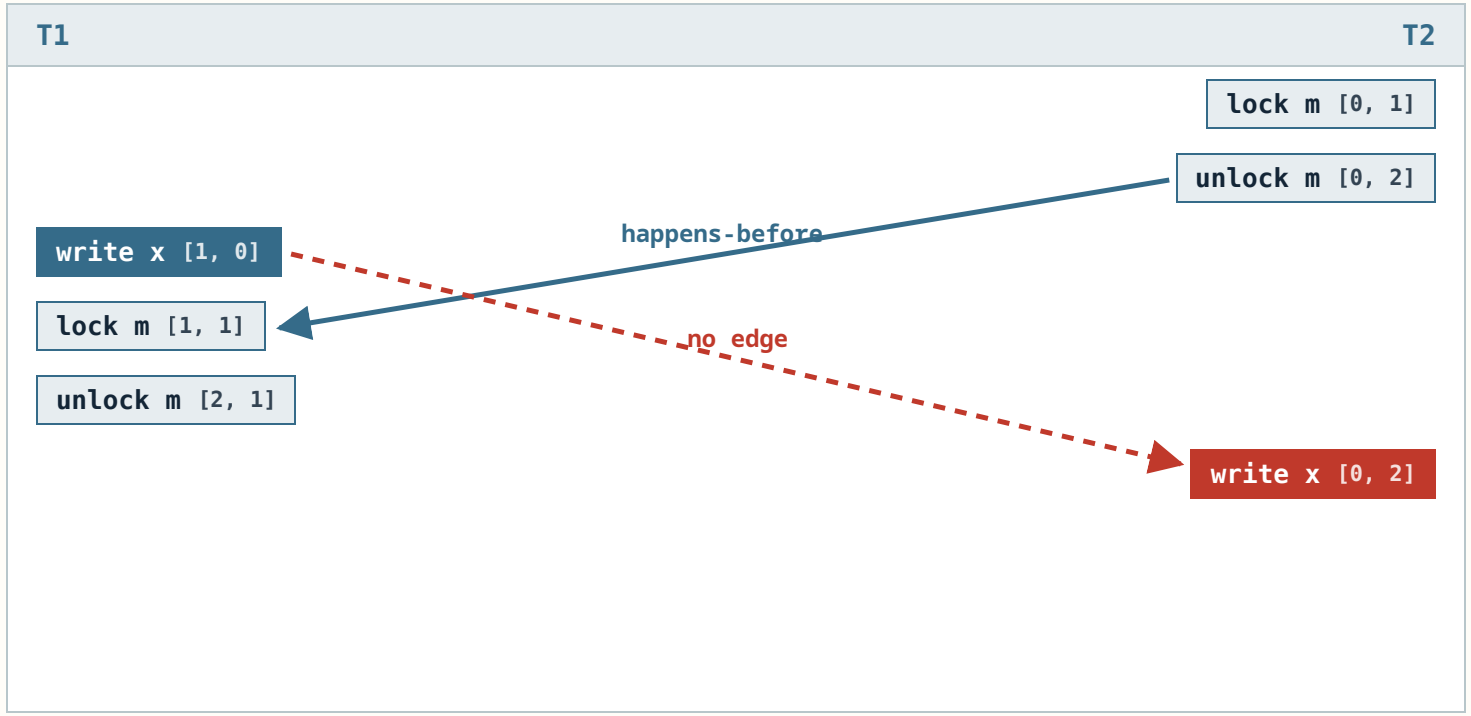
Savage et al. 1997; Flanagan, Freund 2009; Clang TSan docs; Go Data Race Detector docs

Vector clocks step by step

MODEL · VECTOR CLOCKS

T1 clock	T2 clock	m · free	x · last write
[2, 1]	[0, 2]	[1, 1]	W T2@2

T1 T2



DATA RACE: T1@1 > clock[T1]=0

01 FOUR RULES

`[a, b]` = how much of T1 and of T2 this thread has already seen.

`unlock m` → m stores a copy of the clock; the thread ticks its own slot.

`lock m` → clock = max(clock, m): the thread learns what the last owner saw.

`access x` → x remembers `T1@1`. Another thread touches x: is `1 ≤ its clock[T1]`? Yes → ordered. No → **DATA RACE**.

02 SAME CODE, SCENARIO C

```
T1: x = 1;  lock m; unlock m;
T2:         lock m; unlock m;  x = 2;
// m protects something else
```

WHO TAKES M FIRST	TSAN
T1 (default order)	?
T2 (flipped)	1 report

Real TSan run of this program (slide 29): default order 0 reports, flipped 1.

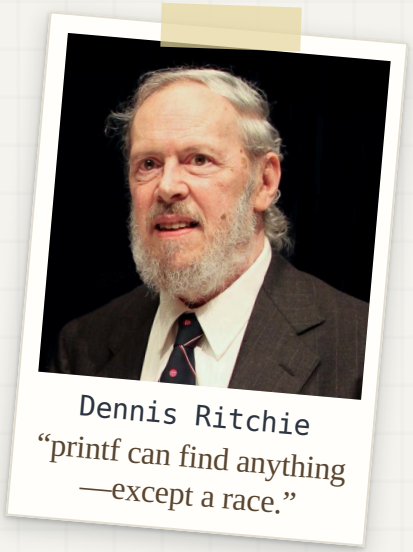
TSan on V0

RESULT

Overdrafts in this run: 0. Races reported: 2.

LINES 33 AND 35

`if (balance >= amt)` and `balance -= amt` access data without a lock. The same lesson as the Go example, now in C++.



Dennis Ritchie
“printf can find anything —except a race.”

RECORDING

```
$ ./bank0_tsan 200 2>&1 | grep -E 'WARNING|SUMMARY|^V='
WARNING: ThreadSanitizer: data race
SUMMARY: ThreadSanitizer: data race bank.cpp:33:9 in Account::withdraw(long)
WARNING: ThreadSanitizer: data race
SUMMARY: ThreadSanitizer: data race bank.cpp:35:15 in Account::withdraw(long)
V=0 runs=200 overdrafts=0 (0.00%)
```

TSan on V1

QUESTION

What will TSan report for V1?

No race report. Every access is ordered by the lock, so happens-before is satisfied. The overdraft still happens.

V1 BANK.CPP

```
{ Guard g(mu); ok = balance >= amt; }
window();
if (ok) { Guard g(mu); balance -= amt; }
```

RECORDING

```
$ ./bank1_tsan 200
V=1 runs=200 overdrafts=1 (0.50%)
```

0

ThreadSanitizer reports

Under TSan, V1 produced 1–5 overdrafts per 200 runs

A deadlock that never happened

TRANSFER.CPP — ONE THREAD

```
void transfer(Account& from, Account& to, long amt) {
  std::lock_guard l1(from.mu);
  std::lock_guard l2(to.mu);
  from.balance -= amt;
  to.balance += amt;
}
int main() {
  Account A, B;
  transfer(A, B, 10);   // edge A -> B
  transfer(B, A, 10);   // edge B -> A
  …
}
```

QUESTION

Only one thread runs, so no deadlock can occur. Will TSan stay silent?

It warns. The observed lock-acquisition orders form a cycle: A → B → A.

RECORDING

```
$ ./transfer_tsan 2>&1 | grep -E 'WARNING|Cycle|^A='
WARNING: ThreadSanitizer: lock-order-inversion (potential deadlock)
  Cycle in lock order graph: M0 (0x7ffe1cc29dc0) => M1 (0x7ffe1cc29d90) => M0
A=100 B=100 (no deadlock in this run)
```

WHAT EACH TOOL SEES

jstack reports an actual deadlock. TSan warns about a potential one after observing both lock orders, even without simultaneous threads.

GO

`go -race` has no lock-order check; an equivalent Go program gets no warning.

-race in CI and L2's blind spots

RECORDING

```
$ go test -race -count=1 .
--- FAIL: TestWithdrawRacy (0.00s)
    testing.go:1865: race detected during execution of test
FAIL
FAIL    bankdemo    0.007s
FAIL
```

HB_MISS.CPP · C++ — SLEEPS THAT SET THE ORDER OMITTED

```cpp
int x = 0;        // shared, never locked
std::mutex m;     // guards something else

// t1                      // t2
x = 1;                    { lock_guard g(m); }
{ lock_guard g(m); }      x = 2;

// default: t1 takes m first    flip: t2 takes m first
```

RECORDING

```
$ ./hb_miss_tsan; ./hb_miss_tsan flip   # TSan report count
default order: reports=0
flipped order: reports=1
```

UBER · PLDI'22

46 million lines of Go; running `-race` in tests found about 2,000 races.

WHAT L2 MISSES

- paths that did not execute
- a race accidentally ordered by an unrelated mutex
- an atomicity violation without a data race (V1)
- waiting on a pool, channel, or Future rather than a mutex
- lock-order inversions in Go

OTHER L2 DETECTORS	+	–
Helgrind, DRD C/C++ · Valgrind	no recompilation; also lock order (Helgrind), long-held locks (DRD)	20–100× slower; custom atomics confuse them
KCSAN Linux kernel	samples accesses: cheap enough for the kernel	kernel only; misses races between samples
Java TSan OpenJDK project	Java and JNI code in one report	experimental; needs a special JDK build



Linus Torvalds
"KCSAN found a race? It may be right. Check the trace."

Take control of the schedule

01

INJECTION

Pause in the gap

With real threads: insert `sleep` or `yield` between check and act.

cheap · schedule not reproducible

02

CUSTOM SCHEDULER

Schedule from a seed

A program switches tasks; a seeded generator chooses their order.

one seed reproduces the bug

03

EXPLORATION

All schedules

Explore every interleaving within a chosen bound.

exhaustive for small tests · state explosion

We will try all three on V1.

Injecting a pause

INJECT.JAVA

```
boolean withdraw(String mode) {
  boolean ok;
  synchronized (this) { ok = balance >= 100; }  // check
  switch (mode) {                               // injection
    case "yield" -> Thread.yield();
    case "sleep" -> park(random(100_000)); // 0–100 µs
    default -> {}
  }
  if (ok) synchronized (this) { balance -= 100; } // act
  return ok;
}
```

RECORDING

```
$ for m in none yield sleep; do java Inject $m 10000; done
none  runs=10,000 overdrafts=3 (0.03%)  1.3s
yield runs=10,000 overdrafts=84 (0.84%)  1.2s
sleep runs=10,000 overdrafts=9,948 (99.48%)  2.6s
```

QUESTION

Without injection, 0.03% fail. Add Thread.yield() between check and act. What happens?

Less than 1%. A 0–100 µs pause yields **99%**.

WHAT HOLDS UP

Order of magnitude: fractions of a percent without injection, about 1% with `yield`, nearly 100% with a pause.

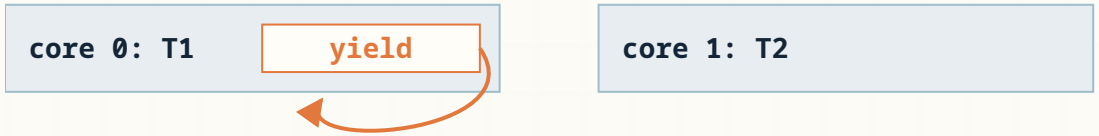
"sleep" uses LockSupport.parkNanos for a random 0–100 µs; CyclicBarrier starts both threads together

Why yield barely helped

01

28 CORES VERSUS ONE

28 CORES



No other runnable thread on this core: T1 continues.
T2 runs on its own core; the window barely grows.

1 CORE · taskset -c 0



yield lets T2 run on the only core;
both checks finish before T1 debits.

injection helps only when it changes the schedule

RECORDING

```
$ taskset -c 0 java Inject yield 10000
yield runs=10,000 overdrafts=9,973 (99.73%)  0.7s
```

SAME YIELD

28 cores: 0.84%. One core: 99.73%.

WHAT CHANGED

`yield` is a hint to the OS, not a forced switch. Testing tools can take control of the schedule instead.

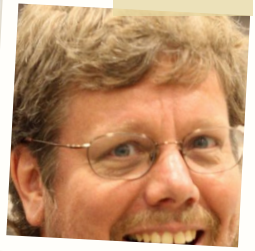
Control the scheduler with a seed

SIM.PY · V1 AS A GENERATOR — PYTHON 3

```python
def withdraw_split(w, tid, amt):
    yield w.acquire("mu")
    ok = w.vars["balance"] >= amt     # check
    yield w.release("mu")
    if not ok: return
    yield w.acquire("mu")
    w.vars["balance"] -= amt          # act
    yield w.release("mu")
# each yield lets the scheduler
# choose who goes next: rng(seed).choice(…)
```

WHY REPLAY WORKS

Every choice of the next task goes through `random.Random(seed)`. Same seed, same schedule, same trace.



Guido van Rossum
"Generator yield is more honest than Thread.yield()"

RECORDING

```
$ python3 sim.py withdraw random 1000
withdraw/random: 529 of 1000 seeds fail (52.9%)
first failing seeds: [0, 1, 3, 5, 6, 8, 10, 13, 14, 18]
```

RECORDING

```
$ python3 sim.py withdraw replay 1
--- replay seed=1, attempt 1
  T0: check balance=100 ok=True
  T1: check balance=100 ok=True
  T1: act   balance=0
  T0: act   balance=-100
  schedule=00111100  -> INVARIANT VIOLATED: balance >= 0  state={'balance': -100}
--- replay seed=1, attempt 2
  T0: check balance=100 ok=True
  T1: check balance=100 ok=True
  T1: act   balance=0
  T0: act   balance=-100
  schedule=00111100  -> INVARIANT VIOLATED: balance >= 0  state={'balance': -100}
```

Enumerate every schedule

2 THREADS × 2 STEPS
= 6 SCHEDULES

MODEL · TWO X++ (LOAD / STORE)

```
A.load  A.store B.load  B.store -> x=2
A.load  B.load  A.store B.store -> x=1  lost update
A.load  B.load  B.store A.store -> x=1  lost update
B.load  A.load  A.store B.store -> x=1  lost update
B.load  A.load  B.store A.store -> x=1  lost update
B.load  B.store A.load  A.store -> x=2
```

6 / 6
schedules checked

4
lost updates

QUESTION

Two `withdraw` calls also take two steps each: six schedules. How many violate the invariant?

4 of 6 — the same count as for **x++**.

RECORDING

```
$ python3 sim.py withdraw dfs; python3 sim.py transfer dfs
withdraw/dfs: explored 6 schedules (all of them), 4 fail
first counterexample schedule=00110011 -> INVARIANT VIOLATED: balance >= 0  st
ate={'balance': -100}
transfer/dfs: explored 6 schedules (all of them), 2 fail
first counterexample schedule=01 -> DEADLOCK: held={'A': 'T0', 'B': 'T1'} wai
ting={'T0': 'B', 'T1': 'A'}
```

DEADLOCK WITHOUT A TIMEOUT

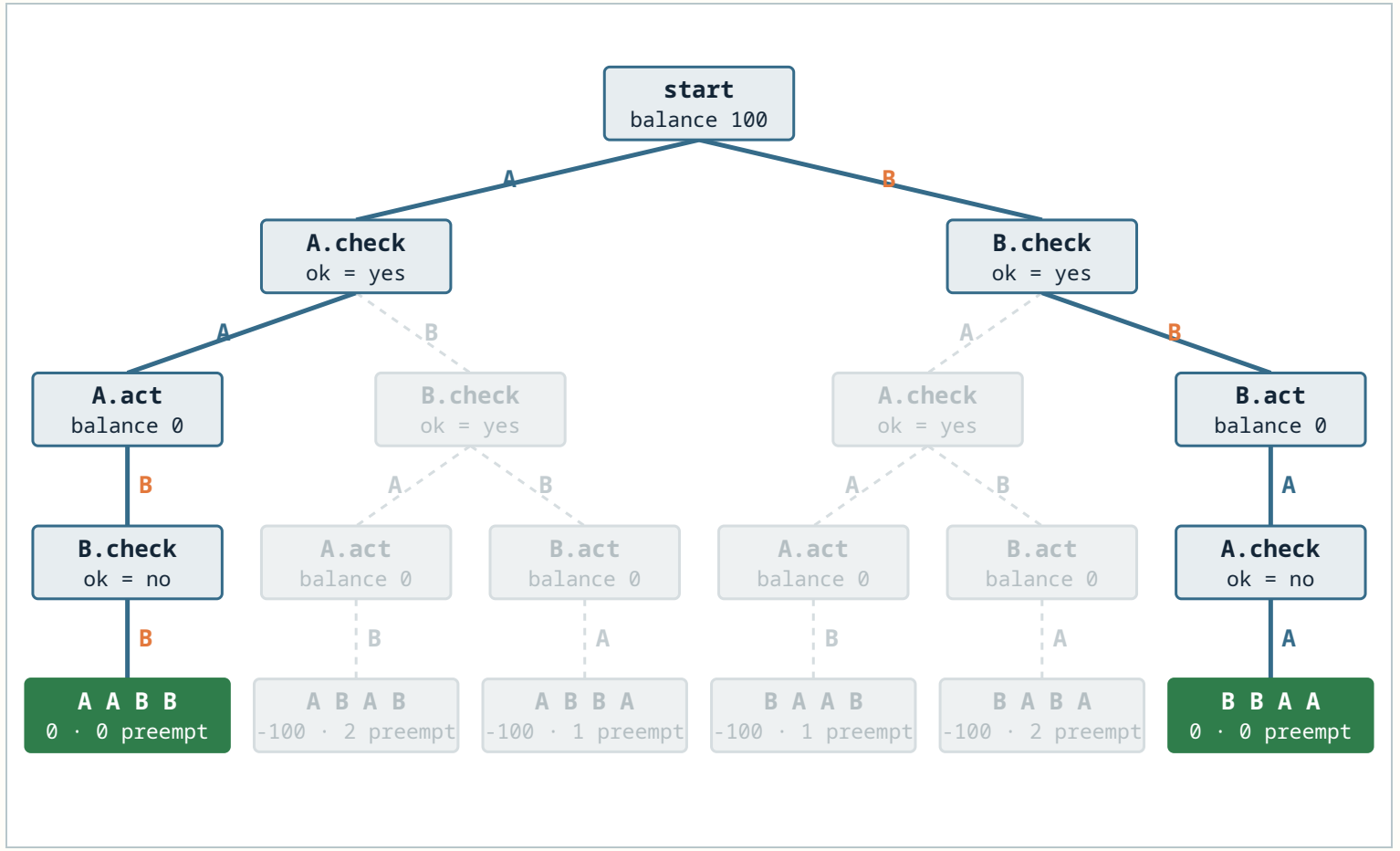
In the simulator, neither thread has an enabled next step. No need to wait: `held={'A': 'T0', 'B': 'T1'}`.

THREE PROGRAMS · EACH THREAD = 2 STEPS — T0 ‖ T1

```
// x++        (model, left)
r = x;  x = r + 1;                    // load · store
// withdraw   (V1, balance 100, both take 100)
lock; ok = balance >= amt; unlock;     // check
lock; if (ok) balance -= amt; unlock;   // act
// transfer   (part 1 deadlock)
T0: lock A; lock B     T1: lock B; lock A
```

The tree of all schedules

MODEL · EVERY BRANCH = WHO RUNS THE NEXT STEP



PREEMPTION BOUND

balance 0 · balance −100 · cut by bound

2 / 2
leaves explored

0
invariant violated

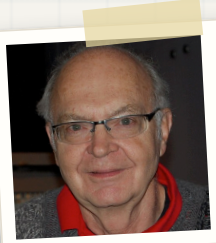
```
bound 0: 2 of 6 schedules left,
no violation among them.
The bug needs 1 preemption.
```

01 PREEMPTION

Switching away from a thread that still has steps left. **A B A B** has 2. **A B B A** has 1: the last switch happens after B is done.

Each withdraw = check (under the lock) + act (under the lock), as V1. Recorded sim.py run: 6 schedules, 4 fail.

Small bounds find real bugs



Donald Knuth
“Premature green tests: another root of evil”

REAL BUGS
OFTEN HAVE SMALL TRACES

01

LU ET AL. · ASPLOS'08

96%

bugs need at most 2 threads

92%

exposed by ordering at most 4 accesses

105 bugs in MySQL, Apache, Mozilla, OpenOffice

02

CHESS · PLDI'07

Enumerate schedules, bounded by preemptions.

≤ 2

preemptions found each of 7 new bugs

An observation on those programs, not a theorem.

03

PCT · ASPLOS'10

Random priorities and d−1 priority-change points.

$$P \geq 1 / (n \cdot k^{d-1})$$

per run: n threads, k steps, bug depth d.

`sim.py withdraw pct 1000` → **25.6%** of seeds find the bug.

TOOL	RUNTIME	WHAT IT DOES
Twist	C++ · MIPT	fault injection and deterministic simulation in Lipovsky’s course
Coyote	.NET	controlled scheduler, PCT
Fray	JVM · OOPSLA'25	controls JVM threads; replay by seed
Shuttle	Rust	randomized schedule testing, PCT
rr chaos	Linux	record and replay; deliberately disruptive schedules

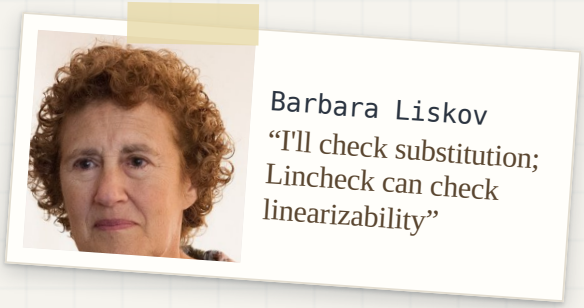
L3 misses: bugs beyond its bounds, nondeterminism outside its scheduler (time, network), and weak-memory behavior.

Lincheck: sequential code as a specification

ACCOUNT.JAVA · V1 — LINCHECK 3.7

```
public class Account {
  private long balance = 100;
  private final Object mu = new Object();

  @Operation
  public boolean withdraw() {           // withdraw 60
    boolean ok;
    synchronized (mu) { ok = balance >= 60; }   // check
    if (!ok) return false;
    synchronized (mu) { balance -= 60; }        // act
    return true;
  }
  …
}
// the entire test:
new ModelCheckingOptions().check(Account.class);
```



01 WHAT IT CHECKS

CONCURRENT	SEQUENTIAL (100 → ?)
T1: withdraw() → true	T1 → true, T2 → false
T2: withdraw() → true	T2 → true, T1 → false

true + true matches neither sequential order → not linearizable

Lincheck invokes operations from different threads. In model-checking mode it instruments bytecode and chooses switch points. The reference is the same class run sequentially.

QUESTION

Every access to `balance` in V1 is synchronized; TSan stayed silent. What will Lincheck report?

A failure: **both withdraw() calls returned true**. No data race, but a specification violation.

How Lincheck works inside

1. **Generate a scenario**
   threads × operations
2. **Instrument the bytecode**
   insert switch points
3. **Run under its own scheduler**
   it decides who runs next
4. **Collect the results**
   what every call returned
5. **Check against sequential runs**
   the reference is your own class
6. **Minimize and report**
   shortest scenario + the schedule

06

WHAT HAPPENS AT THIS STEP

Minimize and report

Lincheck drops calls while the failure persists, then prints the scenario and the interleaving that led to it.

```
= Invalid execution results =
| withdraw(): true | withdraw(): true |

The following interleaving leads to the error:
|   balance → 100  |                  |
|   switch         |                  |
|                  | withdraw(): true |
|   balance = -20  |                  |
```

Play Lincheck: you are the scheduler

Thread 1

withdraw()

```
14  lock mu
14  ok = balance >= 60
14  unlock mu
16  lock mu
16  read balance
16  balance = read - 60
16  unlock mu
    return ok
```

returned true

Thread 2

withdraw()

```
14  lock mu
14  ok = balance >= 60
14  unlock mu
16  lock mu
16  read balance
16  balance = read - 60
16  unlock mu
    return ok
```

returned true

MODEL · ▮▮ = SWITCH POINT INSERTED BY LINCHECK

-20	free	1
BALANCE	LOCK mu	SWITCHES

```
T2  unlock mu
T2  return true
T1  lock mu
T1  read balance → 40
T1  balance = -20
T1  unlock mu
T1  return true
```

Results: T1 → true, T2 → true, balance -20
No sequential order gives true, true. Bug.

WHERE TO SWITCH

Between the two critical sections: after T1 unlocks at line 14, before it locks again at line 16. Lincheck tries such points itself; here the room picks.

The verdict, then minimization

QUESTION

Both calls returned `true`. Is there an order of running them one by one that gives `true, true`?

No. 100 covers only one withdrawal of 60: whoever goes second gets `false`.

01 THE CHECKER

SEEN (concurrent)	T1 → true	T2 → true	
ORDER T1; T2	T1 → true	T2 → false	≠
ORDER T2; T1	T1 → false	T2 → true	≠

No order matches → = Invalid execution results =

02 MINIMIZATION

MODEL · AN ILLUSTRATIVE BIGGER SCENARIO

THREAD 1	THREAD 2
#1 withdraw()	~~#1 balance()~~
~~#2 balance()~~	#2 withdraw()
~~#3 withdraw()~~	~~#3 withdraw()~~

```
scenario: 3 + 3 calls, fails
drop T1 #3 withdraw() → still fails → removed
drop T1 #2 balance()  → still fails → removed
drop T2 #1 balance()  → still fails → removed
drop T2 #3 withdraw() → still fails → removed
drop T2 #2 withdraw() → passes → keep it
```

Remove one call, search the schedules again. Still fails → drop it for good. Passes → put it back.

Lincheck on V1

RECORDING

```
$ java RunLincheck mc
= Invalid execution results =
| ---------------------------------- |
|     Thread 1     |     Thread 2     |
| ---------------------------------- |
| withdraw(): true | withdraw(): true |
| ---------------------------------- |

The following interleaving leads to the error:
| ---------------------------------- |
|     Thread 1     |     Thread 2     |
| ---------------------------------- |
| withdraw(): true |                  |
|   MONITORENTER   |                  |
|   balance → 100  |                  |
|   MONITOREXIT    |                  |
|   switch         |                  |
|                  | withdraw(): true |
|   MONITORENTER   |                  |
|   balance → 40   |                  |
|   balance = -20  |                  |
|   MONITOREXIT    |                  |
|   result: true   |                  |
| ---------------------------------- |
```

RECORDING

```
$ … detailed trace omitted
FAILED (mc)
time: 4.6 s
```

RECORDING

```
$ java RunLincheck fixed
PASSED (fixed)
time: 8.6 s
```

READING THE TRACE

T1 read 100 and released the lock. At `switch`, Lincheck ran T2. T2 withdrew 60; T1 returned and withdrew another 60: `balance = -20`.

The fixed version (check and act in one critical section) passes 10 model-checking iterations.

Lincheck on the JDK: ConcurrentLinkedDeque

RECORDING

```
| ---------------------------------- |

---
All operations above the horizontal line | ----- | happen before those below the line
---

The following interleaving leads to the error:
| ------------------------------------------------------------------------------------------------------------ |
|   Thread 1    |                                          Thread 2                                          |
| ------------------------------------------------------------------------------------------------------------ |
| addLast(1)    |                                                                                            |
| ------------------------------------------------------------------------------------------------------------ |
|               | pollFirst(): 1                                                                             |
|               |   d.pollFirst(): 1                                                                         |
|               |     loop(1 iterations) at ConcurrentLinkedDeque.pollFirst(ConcurrentLinkedDeque.java:915)  |
|               |       <iteration 1>                                                                        |
|               |         first(): Node#2                                                                    |
|               |         loop(2 iterations) at ConcurrentLinkedDeque.pollFirst(ConcurrentLinkedDeque.java:917) |
|               |           <iteration 1>                                                                    |
|               |           <iteration 2>                                                                    |
|               |             p.item → 1                                                                     |
|               |             first.prev → null                                                              |
|               |             switch                                                                         |
| addFirst(-1)  |                                                                                            |
| peekLast(): 1 |                                                                                            |
|               |             p.item.compareAndSet(1, null): true                                            |
|               |     unlink(Node#1)                                                                         |
|               |   result: 1                                                                                |
| ------------------------------------------------------------------------------------------------------------ |
```

WHAT FAILED

`pollFirst` took 1, yet `peekLast` still sees 1 afterward. No sequential order allows that.

RECORDING

```
$ … end of output
time 193.4s
```

Koval et al., CAV'23: Lincheck also found liveness bugs in AQS and in the Kotlin coroutines Mutex.

LINCHECK LIMITS

- one object only: no bugs between services or with the DB
- reference = your class run one call at a time: single-thread bugs pass
- small scenarios, slow search: this run took 193 s
- model checking assumes in-order execution: no weak-memory reorderings
- JVM only

jcstress: how often each outcome occurs

RECORDING

```
$ java org.openjdk.jcstress.Main -t WithdrawTest -m quick
..... [FAILED] demo.WithdrawTest.Broken

  Results across all configurations:

          RESULT     SAMPLES     FREQ      EXPECT  DESCRIPTION
  false, true, 40  59,956,741   46.35%  Acceptable  one withdraw wins
  true, false, 40  66,929,122   51.74%  Acceptable  one withdraw wins
   true, true, -20   2,471,291    1.91%   Forbidden  OVERDRAFT: both passed the check

..... [FAILED] demo.WithdrawTest.Racy

  Results across all configurations:

          RESULT      SAMPLES     FREQ      EXPECT  DESCRIPTION
  false, true, 40  132,415,962   36.96%  Acceptable  one withdraw wins
  true, false, 40  138,064,635   38.54%  Acceptable  one withdraw wins
   true, true, -20            3   <0.01%   Forbidden  OVERDRAFT
    true, true, 40   87,757,874   24.50%   Forbidden  LOST UPDATE: both took money, o
ne write lost
```

WITHDRAWTEST.BROKEN · V1 INSIDE A JCSTRESS HARNESS — JAVA

```java
@JCStressTest
@Outcome(id = {"true, false, 40", "false, true, 40"},
         expect = ACCEPTABLE, desc = "one withdraw wins")
@Outcome(id = "true, true, -20",
         expect = FORBIDDEN,  desc = "OVERDRAFT: both passed the check")
@State public static class Broken {
  long balance = 100;  final Object mu = new Object();
  boolean withdraw() {                    // V1, as on slide 37
    boolean ok; synchronized (mu) { ok = balance >= 60; }
    if (!ok) return false;
    synchronized (mu) { balance -= 60; } return true;
  }
  @Actor   void a1(ZZJ_Result r)    { r.r1 = withdraw(); }
  @Actor   void a2(ZZJ_Result r)    { r.r2 = withdraw(); }
  @Arbiter void check(ZZJ_Result r) { r.r3 = balance; }
}
```

QUESTION

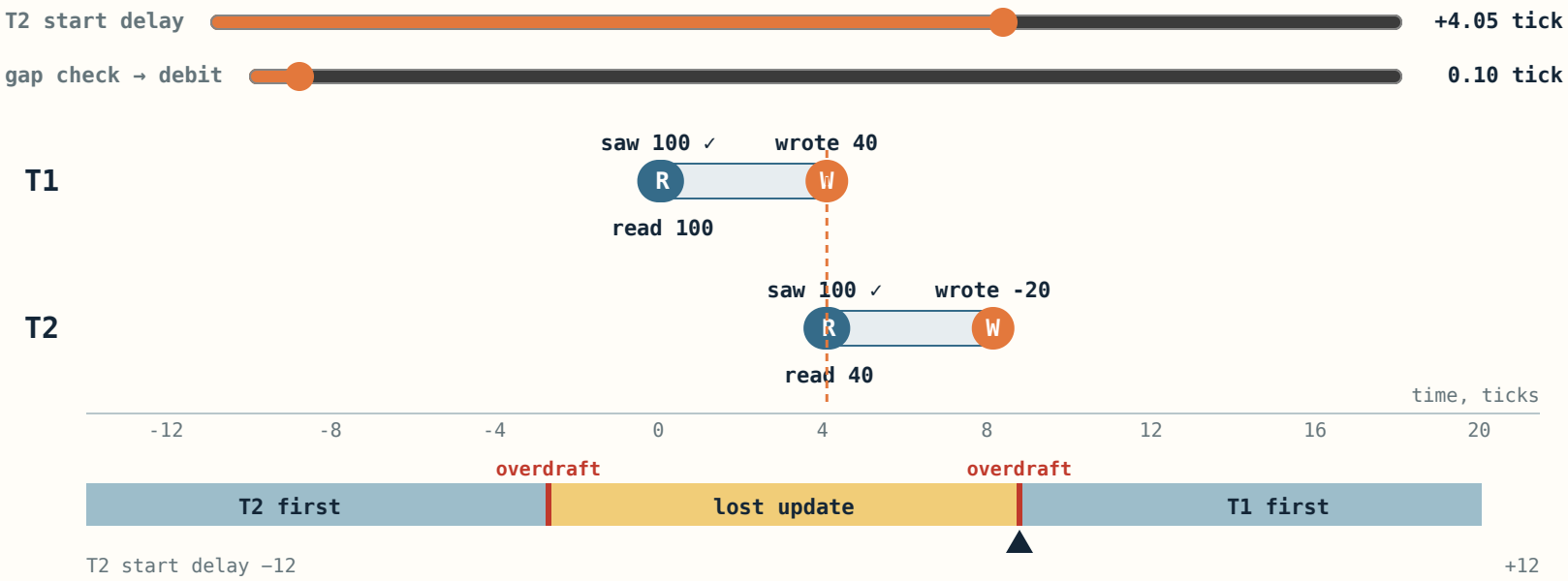
V0 (no lock) has two bugs: overdraft and lost update. Which occurs more often?

Lost update: **24.50%** of outcomes. Overdraft: **3 times** in 358 million.

Lincheck controls the schedule and shows one bad run. jcstress does not: it runs the test millions of times on the real JIT and hardware and counts outcomes.

Why overdraft is rare: the size of the window

MODEL · TIMING OF V0 WITHDRAW, NOT A JVM MEASUREMENT



overdraft

T2 checked 100 before T1 wrote, then debited from 40

r1, r2, balance
true, true, -20

Model, 1,000,000 runs (gap 0.10): one wins 65.84% · lost update 33.34% · **overdraft 0.82%**. Real shares depend on the JIT and hardware; see the measured table.

V0 · NO LOCK — BALANCE = 100

```
if (balance >= 60) {    // C: check
  // gap
  tmp = balance;         // R: read
  balance = tmp - 60; // W: write
}
// balance -= 60 is R + W
```

TWO WINDOWS

Lost update: both read before either writes. Width ≈ 2 × (read → write).

Overdraft: T2 checks before T1 writes, but reads after. Width = the gap between check and debit.

MEASURED · JCSTRESS RUN FROM THE PREVIOUS SLIDE

V0 lost update	**24.50%**
V0 overdraft	**3 in 358M**
V1 overdraft	**1.91%**

TLA+: safety



Leslie Lamport
“First the spec, then the code. Then the spec again.”

WITHDRAW.TLA · PLUSCAL

```
variables balance = Start;

process w \in 1..N
variable ok = FALSE;
begin
  Check: ok := balance >= Amt;
  Act:   if ok then
           balance := balance - Amt;
         end if;
end process;
…
NoOverdraft == balance >= 0
```

LABEL = ATOMIC STEP

Two labels mean two separate critical sections, as in V1. Combine Check and Act under one label and TLC reports “No error has been found”.

Newcombe et al., «How Amazon Web Services Uses Formal Methods», CACM 2015

RECORDING

```
Error: Invariant NoOverdraft is violated.
Error: The behavior up to this point is:
State 1: <Initial predicate>
/\ balance = 100
/\ ok = <<FALSE, FALSE>>
/\ pc = <<"Check", "Check">>

State 2: <Check(1) line 37, col 16 to line 40, col 35 of module Withdraw>
/\ balance = 100
/\ ok = <<TRUE, FALSE>>
/\ pc = <<"Act", "Check">>

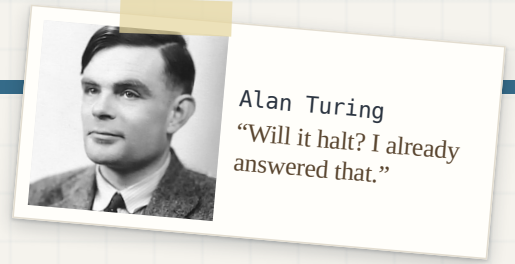
State 3: <Check(2) line 37, col 16 to line 40, col 35 of module Withdraw>
/\ balance = 100
/\ ok = <<TRUE, TRUE>>
/\ pc = <<"Act", "Act">>

State 4: <Act(1) line 42, col 14 to line 48, col 24 of module Withdraw>
/\ balance = 0
/\ ok = <<TRUE, TRUE>>
/\ pc = <<"Done", "Act">>

State 5: <Act(2) line 42, col 14 to line 48, col 24 of module Withdraw>
/\ balance = -100
/\ ok = <<TRUE, TRUE>>
/\ pc = <<"Done", "Done">>

13 states generated, 12 distinct states found, 3 states left on queue.
```

TLA+: liveness as a cycle



POLITE.TLA · PLUSCAL

```
variables flag = <<FALSE, FALSE>>;
                    \* "I want in"
fair process p \in {1, 2}
                    \* fair = weak fairness
begin
Try:  flag[self] := TRUE;
Look: if flag[3 - self]
      then goto Back;
      else goto CS; end if;
Back: flag[self] := FALSE;
      goto Try;
CS:   skip;
end process;

SomeoneEnters ==
  <>(\E i \in {1, 2} :
       pc[i] = "CS")
```

<> = eventually. Reconstructed spec; line numbers in the trace refer to the TLA+ translation.

RECORDING

```
$ java tlc2.TLC Polite   # polite pair
Error: Temporal property SomeoneEnters was violated.

Error: The following behavior constitutes a counter-example:

State 1: <Initial predicate>
/\ flag = <<FALSE, FALSE>>
/\ pc = <<"Try", "Try">>

State 2: <Try(1) line 37, col 14 to line 39, col 50 of module Polite>
/\ flag = <<TRUE, FALSE>>
/\ pc = <<"Look", "Try">>

State 3: <Try(2) line 37, col 14 to line 39, col 50 of module Polite>
/\ flag = <<TRUE, TRUE>>
/\ pc = <<"Look", "Look">>

State 4: <Look(1) line 41, col 15 to line 45, col 29 of module Polite>
/\ flag = <<TRUE, TRUE>>
/\ pc = <<"Back", "Look">>

State 5: <Look(2) line 41, col 15 to line 45, col 29 of module Polite>
/\ flag = <<TRUE, TRUE>>
/\ pc = <<"Back", "Back">>

State 6: <Back(2) line 47, col 15 to line 49, col 50 of module Polite>
/\ flag = <<TRUE, FALSE>>
/\ pc = <<"Back", "Try">>

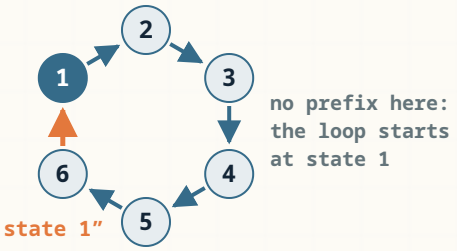
Back to state 1: <Back(1) line 47, col 15 to line 49, col 50 of module Polite>
```

QUESTION

What will TLC find in the TLA+ model of the polite pair: mutual-exclusion failure, deadlock, or nothing?

A liveness violation: “someone eventually enters” fails even under a fair scheduler.

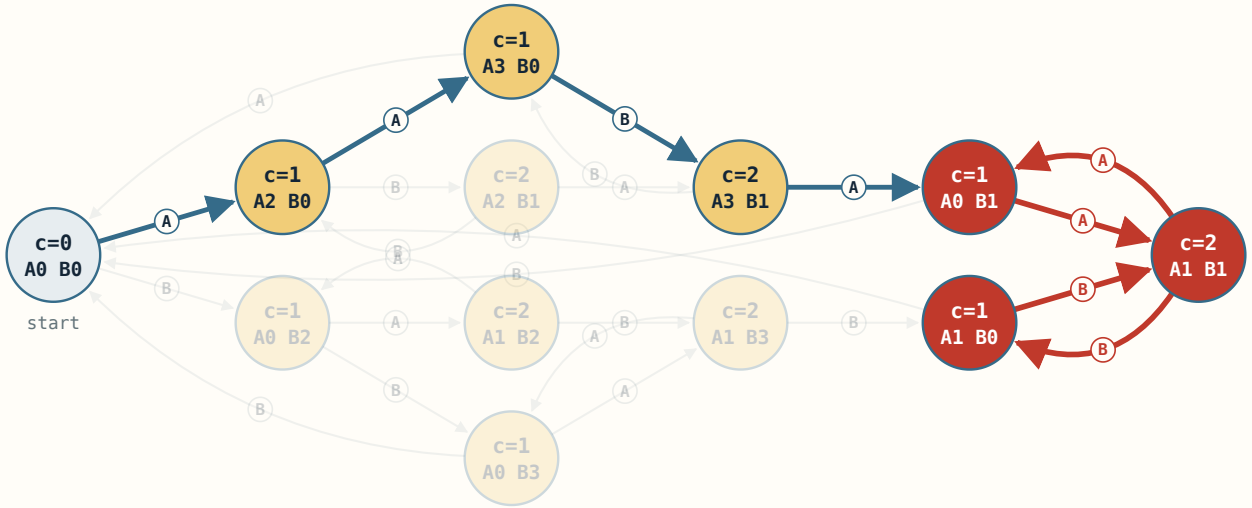
THE WITNESS IS A CYCLE



A race has a finite witness; livelock has a cycle. Weak fairness: a step that stays enabled eventually runs.

How a model checker walks the state graph

MODEL · VISIT EVERY STATE OF THE SPINLOCK, LAYER BY LAYER



12 / 12	0	AABA → (ABBA)∞
states found	new in this layer	path, then a loop: livelock

Back to the same state, forever: A B B A ∞. Both threads keep moving, nobody enters: livelock. 12 states in total.

```
SPINLOCK        LINE NO. = WHERE A THREAD IS

     0 while (c.fetch_add(1) > 0)
       +1; was it already > 0? → busy

     1   c.fetch_sub(1);
       back off: −1, go to line 0

     2 /* critical section */
       holds the lock

     3 c.fetch_sub(1);
       unlock: −1, go to line 0
```

One circle = one state: the counter `c` and the line each thread is on, e.g. `A0 B1`.

Arrow A or B = one step of that thread.

- someone holds the lock
- new in this layer
- path from the start
- loop where nobody enters

DST in production systems

Same idea as slide 33, for a whole cluster: time, network, disk, and scheduling go through one seeded simulator.

01

FOUNDATIONDB

Cluster in one process

A deterministic simulation of the whole cluster, machines, drives, and network, in a single-threaded process. Machine, network, and datacenter failures are injected; a simulated run repeats exactly.

`apple.github.io/foundationdb/testing.html`

02

TIGERBEETLE VOPR

Liveness mode, 2023

Found a liveness bug: replica A, missing two entries, sent repair requests round-robin and always asked each peer for the entry that peer lacked.

`"Simulation Testing for Liveness", Jul 6, 2023`

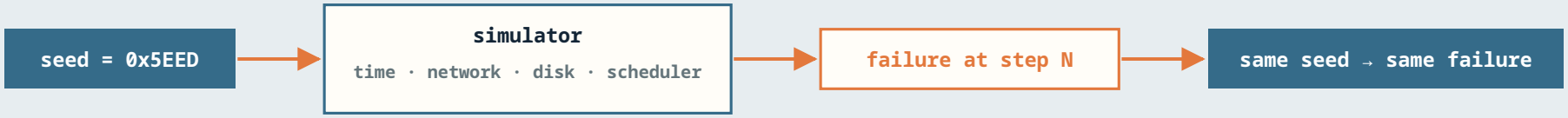
03

ANTITHESIS

Deterministic hypervisor

Your whole system, with dependencies and clients, runs in a fully deterministic environment with injected faults. Every bug found is reproducible.

`antithesis.com/docs, "How Antithesis works"`



CONSTRAINT

An untracked clock read or thread can break deterministic replay. To find livelock, the simulator must also check for progress, not merely failures.

OUTSIDE THE SYSTEM

Jepsen (with Knossos, Elle) and Porcupine check recorded operation histories rather than control scheduling. Porcupine was built for the MIT 6.5840 key-value labs.

In production: two thread dumps and JFR

01

TWO THREAD DUMPS, 5 SECONDS APART

STACKS	CPU	PROGRESS	POSSIBLE CAUSE
unchanged	~0	none	**deadlock or external wait**
changing	high	none	**possible livelock**
same lock	low	slow	**contention**

Clues, not a diagnosis: check the wait dependency and progress metrics.

QUESTION

16 threads, 4,739 requests/s, low CPU. Where does the time go?

Waiting for a lock: **3,363** monitor-contention events at `MetricsCollector.log`, averaging **17.7 ms**. One shared `synchronized` method sits on the hot path.

Useful JFR events: `jdk.JavaMonitorEnter`, `jdk.ThreadPark`, `jdk.VirtualThreadPinned`. async-profiler `-e lock` can show the contended stack.

HOT.JAVA

```
synchronized void log(String event) {  // shared lock
  count++;
  long until = System.nanoTime() + 200_000; // 0.2 ms
  while (System.nanoTime() < until) { }
}
```

RECORDING

```
$ java -XX:StartFlightRecording=…,settings=profile Hot; jfr view contention-by-site
threads=16 requests/s=4,739

                                Contention by Site

StackTrace                                                  Count    Avg.    Max.
----------------------------------------------------------- ----- ------- -------
Hot$MetricsCollector.log(String)                            3,363 17.7 ms 37.9 ms
```

Goroutine leaks and off-CPU time



MAIN.GO — 100 REQUESTS · 10 MS TIMEOUT

```go
func handleRequest() error {
    ch := make(chan error) // unbuffered
    go func() { ch <- doHeavyWork() }()   // line 17
    select {
    case err := <-ch: return err
    case <-time.After(10 * time.Millisecond):
        return errors.New("timeout")
    }
}
…
pprof.Lookup("goroutineleak").WriteTo(os.Stdout, 1)
```

RECORDING

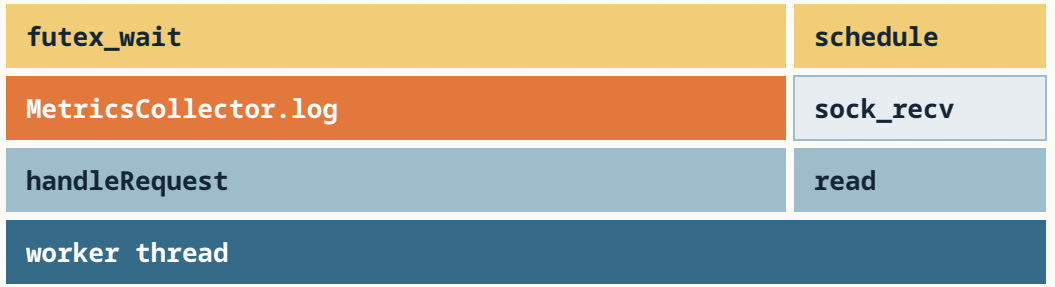
```
$ go run main.go
goroutineleak profile: total 100
100 @ 0x47f06a 0x41421c 0x413e17 0x4e043a 0x484ec1
#   0x4e0439    main.handleRequest.func1+0x39    .../go/leak/main.go:17
```

The leak profile reports goroutines stuck at the send site. In tests, **goleak** can check whether goroutines remain after a test finishes.

01 OFF-CPU FLAME GRAPH · DIAGRAM

width = time waiting, not CPU time

stack captured when thread leaves CPU



Schematic. Real capture: bcc **offcputime** (needs root).

WHY OFF-CPU?

A CPU profile misses sleeping threads. Off-CPU profiling shows where they wait: a lock, a socket, or disk I/O.

Which tools found V1?

LEVEL	TOOL	V0 RACE	V1 TWO SECTIONS	V2
L1	Clang TSA, ErrorProne	**compile error**	**silent**	—
L2	TSan, `go test -race`	**DATA RACE**	**silent; 1/200 overdrafts**	—
L3	delay injection	—	**99.48% of runs fail**	—
L3	`sim.py`, exhaustive search	—	**4 of 6 schedules**	—
L4	Lincheck	—	**trace in 4.6 s**	**PASSED in 8.6 s**
L4	jcstress	**LOST UPDATE 24.50%**	**OVERDRAFT 1.91%**	—
L5	TLA+, TLC	—	**5-state counterexample**	—

QUESTION

Which levels caught V1?

Tools that **control schedules** (L3) or **check a specification** (L4, L5). Static access checks and race detectors stay silent: every V1 memory access is under a lock.

BLIND SPOTS

- TSan uses happens-before, not lockset analysis.
- `go -race` does not report lock-order inversion; C++ TSan can.
- DST needs a liveness check to find livelock.
- Coyote targets .NET.
- eBPF, rr chaos, and Fray: not shown live.

What to use in your own project

WHEN	JAVA	C++
L1 while writing code every build	ErrorProne with `@GuardedBy`	Clang `-Wthread-safety` with `GUARDED_BY` / `absl::Mutex`
L2 every test run in CI catches data races	no standard race detector; Java TSan is an experimental OpenJDK build	`-fsanitize=thread` on the test suite (5–15× slower)
L3 a suspected window check-then-act, retries	inject a delay into the window; Fray for controlled schedules (not shown live)	inject a delay into the window; `rr record --chaos` (not shown live)
L4 your own concurrent class counter, cache, queue	Lincheck in unit tests; jcstress for lock-free / memory-model code	stress test with an invariant check under TSan; no Lincheck-like tool shown today
L5 a protocol or algorithm before writing it	TLA+ / TLC on a small model: 2–3 processes, small values	
L6 production hangs, slowdowns	2 × `jstack`; JFR (`JavaMonitorEnter`, `VirtualThreadPinned`); `async-profiler -e lock`	2 × `gdb -p PID -ex "thread apply all bt"`; `perf lock contention`; off-CPU profile
always wrong data, no hang	assert invariants in code, alert on them in production (`balance < 0` → alert)	

Same bugs, everyday code



Rob Pike
“No channels today. Same bugs anyway.”

JAVA · C++
4 CASES

PART 3

Recognize the bug

01 **See the code. Predict the behavior.**

02 **Name the bug from Part 1.**

03 **Watch the run. Choose a fix.**

FOUR CARDS

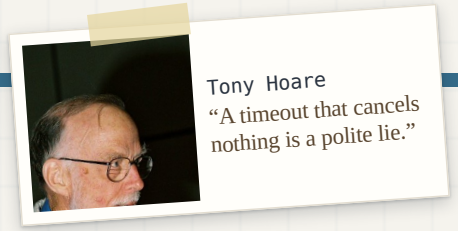
- Java · a request timeout and a thread pool
- C++ · a signal and a waiting thread
- Java · one worker thread for everything
- Java · a queue between two threads

ANSWERS TO CHOOSE FROM

data race · deadlock · livelock · starvation · lock convoy · pool exhaustion · waiting with nobody to wake you

All recordings are real runs: JDK 21, clang 22, Linux.

The request gave up. The task did not.



POOLTIMEOUT.JAVA — JDK 21

```java
var pool = Executors.newFixedThreadPool(4);  // 4 workers

for (int i = 0; i < 100; i++) {           // 100 requests
  var f = pool.submit(() -> backend(i));
    // backend: 20 ms, but every 10th call takes 2 s
  try {
    f.get(50, MILLISECONDS);   // wait at most 50 ms
    ok++;
  } catch (TimeoutException e) {
    timeout++;                 // give up and move on…
    // …but the task keeps running
  }
}
```

QUESTION

Only every 10th call is slow. Out of 100 requests, how many succeed?

27. Abandoned 2-second tasks keep all four workers busy, so fast requests wait in the queue and time out too. Fix: `f.cancel(true)` on timeout frees the worker.

RECORDING

```
$ java PoolTimeout
after  20 requests: ok= 18  timeout=  2
after  40 requests: ok= 27  timeout= 13
after  60 requests: ok= 27  timeout= 33
after  80 requests: ok= 27  timeout= 53
after 100 requests: ok= 27  timeout= 73
```

RECORDING

```
$ java PoolTimeout fix   # f.cancel(true) on timeout
after  20 requests: ok= 18  timeout=  2
after  40 requests: ok= 36  timeout=  4
after  60 requests: ok= 54  timeout=  6
after  80 requests: ok= 72  timeout=  8
after 100 requests: ok= 90  timeout= 10
```

With the fix, exactly the 10 slow calls time out; the other 90 succeed.

The signal came before anyone listened

LOST_WAKEUP.CPP C++17

```cpp
std::mutex m;
std::condition_variable cv;   // lets one thread wake another
bool ready = false;

// producer
{ std::lock_guard g(m); ready = true; }
cv.notify_one();              // "wake up, data is ready"

// consumer (starts 100 ms later)
std::unique_lock lk(m);
cv.wait(lk);                  // sleep until notified
use_data();
```

RECORDING

```
$ ./lost_wakeup   # wait gives up after 2 s so the demo ends
t=   0 ms  producer: ready = true, notify_one()
t= 100 ms  consumer: starts waiting
t=2100 ms  consumer: still waiting, nobody will notify again
```

RECORDING

```
$ ./lost_wakeup fix   # wait(lk, []{ return ready; })
t=   0 ms  producer: ready = true, notify_one()
t= 100 ms  consumer: starts waiting
t= 100 ms  consumer: woke up, ready = 1
```

QUESTION

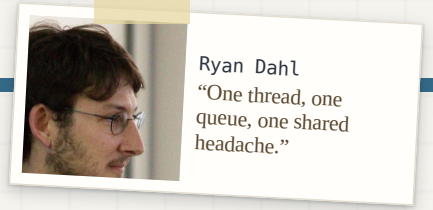
The producer has already set `ready = true`. Will the consumer wake up?

No, never. `notify_one()` is not stored: it woke nobody, because nobody was waiting yet. Fix: `cv.wait(lk, []{ return ready; })` checks the flag first and does not sleep at all.

RULE

Never wait without a condition. Check the flag under the lock, then sleep only if it is false.

One worker thread for everything



PING EVERY 250 MS
REPORT = 2 S OF CPU WORK

SINGLEEXECUTOR.JAVA — JDK 21

```java
var worker = Executors.newSingleThreadExecutor();
                              // ONE thread, tasks run in turn

// a client sends a ping every 250 ms
worker.submit(() -> answerPing());    // takes ~0 ms

// once, someone asks for a report
worker.submit(() -> heavyReport());  // 2 s of CPU
```

QUESTION

While the report runs, how long does a ping wait: 2 ms, 20 ms, or 2 s?

About 2 s. The only thread is busy with the report; every ping queues behind it. Fix: give heavy work its own executor.

SAME SHAPE ELSEWHERE

A GUI thread, a Node.js event loop, a Redis server: one thread, one queue. One slow task makes everyone wait.

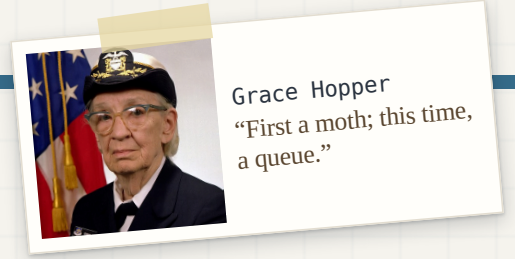
RECORDING

```
$ java SingleExecutor
t=   0 ms  ping waited     1 ms
t= 251 ms  ping waited     0 ms
t= 502 ms  >>> report submitted (2 s of CPU work)
t= 502 ms  ping waited  2001 ms  <-- STUCK
t= 754 ms  ping waited  1750 ms  <-- STUCK
t=1004 ms  ping waited  1500 ms  <-- STUCK
t=1254 ms  ping waited  1250 ms  <-- STUCK
t=1504 ms  ping waited  1000 ms  <-- STUCK
t=1754 ms  ping waited   750 ms  <-- STUCK
```

RECORDING

```
$ java SingleExecutor fix   # report on a separate executor
t=   0 ms  ping waited     1 ms
t= 251 ms  ping waited     0 ms
t= 502 ms  >>> report submitted (2 s of CPU work)
t= 502 ms  ping waited     3 ms
t= 755 ms  ping waited     0 ms
t=1005 ms  ping waited     0 ms
t=1255 ms  ping waited     0 ms
```

Unbounded queue



PRODUCER 10,000/S · CONSUMER 2,000/S
-XMX64M · 2 KB PER MESSAGE

BACKPRESSURE.JAVA — JDK 21

```
BlockingQueue<byte[]> q = bounded
    ? new ArrayBlockingQueue<>(1000)
    : new LinkedBlockingQueue<>();   // unbounded
// producer: 100 put() calls every 10 ms
// consumer: 20 take() calls every 10 ms
```

QUESTION

How long before the unbounded `LinkedBlockingQueue` exhausts memory: a minute, an hour, or never?

About four seconds in this 64 MB run. Bound the queue: `put()` then waits when full, slowing the producer to the consumer's rate.

RECORDING

```
$ java -Xmx64m Backpressure unbounded
t= 1s  produced=  9,800  in queue=  7,820  heap= 22 MB
t= 2s  produced= 19,300  in queue= 15,380  heap= 34 MB
t= 3s  produced= 28,900  in queue= 23,020  heap= 51 MB
t= 4s  produced= 38,100  in queue= 30,320  heap= 62 MB
java.lang.OutOfMemoryError at t=4.1s, produced 38,993 messages
```

RECORDING

```
$ java -Xmx64m Backpressure bounded
t= 1s  produced=  3,000  in queue=    980  heap=  8 MB
t= 2s  produced=  5,000  in queue=    980  heap= 13 MB
t= 3s  produced=  6,900  in queue=    980  heap= 16 MB
t= 4s  produced=  8,900  in queue=    980  heap= 20 MB
t= 5s  produced= 10,800  in queue=    980  heap=  4 MB
t= 6s  produced= 12,800  in queue=    980  heap=  8 MB
alive after 6 s, queue=980
```

AWS postmortem, October 2025

US-EAST-1 · DYNAMODB DNS
19.10 23:48 – 20.10 14:20 PDT

PES

EXCERPTS FROM THE POST-EVENT SUMMARY (VERBATIM)

01 «Before it begins to apply a new plan, the DNS Enactor makes a one-time check that its plan is newer than the previously applied plan.»

02 «…one DNS Enactor experienced unusually high delays needing to retry its update on several of the DNS endpoints.»

03 «…the DNS Planner continued to run and produced many newer generations of plans.»

04 «When the second Enactor (applying the newest plan) completed its endpoint updates, it then invoked the plan clean-up process…»

05 «…the first Enactor (which had been unusually delayed) applied its much older plan to the regional DDB endpoint, overwriting the newer plan.»

06 «…resulted in an incorrect empty DNS record for the service's regional endpoint (dynamodb.us-east-1.amazonaws.com) that the automation failed to repair.»

aws.amazon.com/message/101925/ · Summary of the Amazon DynamoDB Service Disruption in the Northern Virginia (US-EAST-1) Region

IN PAIRS · 2 MINUTES

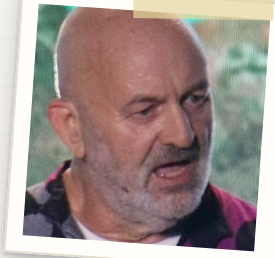
01 Where is the check?

02 Where is the act?

03 Which tool from Part 2 might have caught this before production?

TIMER

2:00



Werner Vogels
Everything fails all the time. Especially a stale "my plan is newer" check.

AWS: minutes between check and act

1 · check

DNS Enactor 1 checked once that its plan was newer, then began updating endpoints. Its transactions encountered conflicts and repeated delays; the check became stale.

2 · window

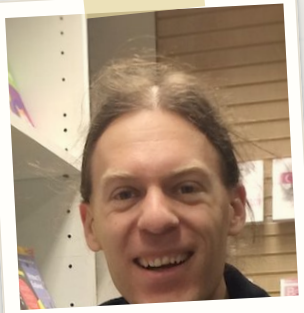
The Planner produced many newer plan generations. Enactor 2 applied the latest plan and started cleanup of older plans.

3 · act

Enactor 1 overwrote the newer regional endpoint with its stale plan. Cleanup removed that plan's endpoints: the DNS record for `dynamodb.us-east-1.amazonaws.com` became empty. Automation did not repair it; engineers intervened.

4 · EC2

EC2's DropletWorkflow Manager could not renew host leases fast enough. Recovery work piled up; AWS called it “congestive collapse … unable to make forward progress.” Engineers intervened to restore progress.



Martin Kleppmann
“Check-then-act, now with network latency.”

STEPS 1–3

The same check-then-act pattern as slide 5. Here the gap lasts minutes, not nanoseconds.

COULD THIS BE CAUGHT BEFORE PRODUCTION?

A protocol model or deterministic simulation could inject a delay between the check and the update—if the model includes this stale-plan case. Neither tool catches it automatically.

STEP 4

It resembles livelock by symptom: activity without progress. AWS called it congestive collapse, not livelock. Watch useful-work metrics.

Five things to do this week

R1 Run relevant tests regularly with `-race` or `-fsanitize=thread` in CI.
slides 4, 26, 29

R2 Make check-then-act on shared state atomic: `merge`/`compute`, CAS, a transaction, or `UPDATE … WHERE`.
slides 5, 59

R3 Avoid waiting inside a pool for work queued to that same pool. Bound queues and concurrency.
slides 8, 54, 57

R4 Measure useful work in every background loop and service. Alert when CPU is busy but progress stops.
slides 12, 56

R5 Test custom concurrent structures with Lincheck or inject delays at suspected interleavings. Add backoff and jitter to retries.
slides 13, 31, 41



Ada Lovelace
“No hardware, no flaky tests.”

Find the bad schedule in CI before your users find it in production.

Symptom → bug → tool → fix

SYMPTOM	BUG	HOW TO FIND IT	HOW TO FIX IT
Results vary; a test is flaky	**data race**	TSan, `go -race`; Clang TSA or ErrorProne with annotations	atomic operation or mutex
A thread-safe collection loses updates	**check-then-act**	delay injection, Lincheck, jcstress, TLA+	one atomic operation: `merge`/`compute`, CAS, or one critical section
Stalled, CPU ≈ 0%; jstack finds a cycle	**lock deadlock**	jstack (actual); TSan lock-order (potential; Go `-race` lacks it)	consistent lock order
Stalled, CPU ≈ 0%; jstack finds no deadlock	**waiting on a pool, `Future`, or channel**	compare thread dumps; in Go: goroutine dump, goroutineleak profile, `goleak`	avoid self-wait in a pool; for channels use `select` with `ctx.Done()` or ensure a receiver. Buffering alone only defers the bug
CPU busy, threads RUNNABLE, useful work ≈ 0	**livelock**	progress metric; TLA+ with fairness (lasso); DST with a liveness check	break symmetry: priority, queue, or jitter
One thread or client fares far worse	**starvation**	per-thread acquisition counts, tail latency; `tryLock(timeout)` as a probe	FIFO/fair lock (throughput cost), shorter critical section, avoid immediate reacquisition
Fair lock enabled; throughput fell by 3–5×	**lock convoy**	context switches per acquisition; JFR, async-profiler `-e lock`	shorter critical section, sharding, fairness only where required
A thread waits forever though the event already happened	**lost wakeup**	thread dump: stuck in `wait`; nobody left to notify	wait with a condition: `cv.wait(lk, pred)` / `while (!ready) wait()`
Memory grows; producer outpaces consumer	**missing backpressure**	queue depth, heap	bounded queue

Questions

Questions?

A test checks one schedule; a bug lives in another.

Let tools find it before production does.

src

SOURCES

- Lu et al. «Learning from Mistakes». ASPLOS 2008
- Tu et al. «Understanding Real-World Concurrency Bugs in Go». ASPLOS 2019
- Chabbi, Ramanathan. «A Study of Real-World Data Races in Golang». PLDI 2022 (Uber)
- Musuvathi, Qadeer et al. CHESS. OSDI 2008 · Burckhardt et al. PCT. ASPLOS 2010
- Koval et al. «Lincheck». CAV 2023
- Newcombe et al. «How Amazon Web Services Uses Formal Methods». CACM 2015
- AWS Post-Event Summary, us-east-1, 19–20.10.2025 · aws.amazon.com/message/101925/
- Cloudflare. Details of the outage of July 2, 2019
- JEP 444 Virtual Threads · JEP 491 Synchronize Virtual Threads without Pinning

ph

PHOTOS: WIKIMEDIA COMMONS

Ken Thompson — A.C.Diller, CC BY-SA 4.0
James Gosling — Peter Campbell, CC BY-SA 4.0
Edsger Dijkstra — Hamilton Richards, CC BY-SA 3.0
Andrew Tanenbaum — Jantangring, CC BY-SA 4.0
Brian Kernighan — Ben Lowe, CC BY 2.0
Butler Lampson — Duncan.Hull, CC BY-SA 4.0
Bjarne Stroustrup — Julia Kryuchkova, CC BY-SA 2.5
Dennis Ritchie — Denise Panyik-Dale, CC BY 2.0
Linus Torvalds — Krd (photo), Von Sprat (crop), CC BY-SA 4.0
Guido van Rossum — Doc Searls (photo), crop: Maartenschrijft, CC BY-SA 2.0
Donald Knuth — Alex Handy, CC BY-SA 2.0
Barbara Liskov — Kenneth C. Zirkel, CC BY-SA 3.0
Leslie Lamport — Leslie Lamport, Copyrighted free use
Alan Turing — Elliott & Fry, Public domain
Brendan Gregg — xchiamiov, CC BY 2.0
Rob Pike — Chlor at English Wikipedia, CC BY-SA 3.0
Tony Hoare — Dennis Hamilton, CC BY 2.0
Ryan Dahl — David Calhoun, CC BY 2.0
Grace Hopper — James S. Davis, Public domain
Werner Vogels — Xuthoria, CC BY-SA 4.0
Martin Kleppmann — Martin Kleppmann, CC BY 3.0
Ada Lovelace — Alfred Edward Chalon, Public domain

Sticker captions are jokes, not quotations.